POLITEKNIK NEGERI BENGKALIS
Jurusan Teknik Informatika · Prodi D4 Rekayasa Perangkat Lunak

JOBSHEET PRAKTIKUM
Pengenalan PHPUnit

Mata Kuliah	Pengujian dan Pemeliharaan Perangkat Lunak
Topik	Pengenalan unit testing dengan PHPUnit
Pertemuan ke-	
Alokasi Waktu	3 × 50 menit
Dosen Pengampu	Muhammad Asep Subandri, M.Kom

Nama	Rio dinata
NIM	6304240053
Kelas	5B

A. Tujuan Praktikum

1. Memasang PHPUnit pada proyek PHP menggunakan Composer.
2. Menulis unit test dengan pola Arrange-Act-Assert dan assertion dasar.
3. Menjalankan test serta membaca hasil lulus maupun gagal.
4. Menggunakan data provider untuk menjalankan satu test dengan banyak data.

B. Dasar Teori Singkat

Unit testing adalah pengujian otomatis terhadap bagian terkecil kode, biasanya satu method atau satu class, secara terpisah dari bagian lain. Setiap test disusun dengan pola **Arrange-Act-Assert (AAA)**: siapkan objek dan data (Arrange), jalankan perilaku yang diuji (Act), lalu bandingkan hasilnya dengan harapan (Assert).

PHPUnit adalah framework unit testing standar untuk PHP. Konvensinya: class test berakhiran `Test` dan mewarisi `PHPUnit\Framework\TestCase`; method test bersifat public, diawali kata `test`, dan mengembalikan `void`; hasil diperiksa dengan method assertion seperti `assertSame($harapan, $hasil)`.

Assertion	Memeriksa
`assertSame()`	Nilai dan tipe identik (===). Gunakan sebagai pilihan utama.
`assertEquals()`	Nilai sama (==), tipe boleh berbeda.
`assertTrue()` / `assertFalse()`	Nilai boolean.
`assertStringContainsString()`	Sebuah teks memuat potongan teks tertentu.
`assertInstanceOf()`	Objek bertipe class tertentu.

Data provider adalah method public static yang mengembalikan array data uji. Method test yang ditandai attribute #[DataProvider('namaMethod')] dijalankan sekali untuk setiap baris data.

Contoh kode pada jobsheet ini diadaptasi dari contoh resmi PHPUnit: class Greeter dari manual PHPUnit (bab Writing Tests for PHPUnit) dan class Email dari panduan Getting Started di phpunit.de.

C. Alat dan Bahan

Perangkat	Keterangan
PHP 8.2 atau lebih baru	Cek dengan php -v. Jobsheet ini memakai PHPUnit 11.
Composer 2.x	Cek dengan composer --version.
Text editor	Visual Studio Code atau editor lain.
Terminal dan koneksi internet	Untuk menjalankan perintah dan mengunduh PHPUnit.

Catatan untuk pengguna Windows

Di Command Prompt, tulis vendor\bin\phpunit sebagai ganti vendor/bin/phpunit. Di PowerShell dan Git Bash keduanya bisa dipakai.

D. Langkah Praktikum

Bagian	Kegiatan	Waktu
Praktikum 1	Memasang PHPUnit	25 menit
Praktikum 2	Test pertama: Greeter	25 menit
Praktikum 3	Beberapa assertion: Email	25 menit
Praktikum 4	Data provider	30 menit
Latihan	Latihan mandiri	45 menit

Praktikum 1 – Memasang PHPUnit (25 menit)

1. Buat folder proyek, masuk ke dalamnya, lalu buat composer.json dan pasang PHPUnit:

Terminal

```
mkdir belajar-phpunit
cd belajar-phpunit
composer init --name=polbeng/belajar-phpunit -n
composer require --dev phpunit/phpunit:^11.5
```

2. Buka composer.json dan tambahkan bagian autoload berikut (sejajar dengan require-dev). Dengan classmap, semua class di folder src/ bisa langsung dipakai tanpa require:

composer.json (tambahan)

```
"autoload": {
    "classmap": ["src/"]
}
```

3. Buat folder src dan tests, lalu buat file phpunit.xml di root proyek:

phpunit.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<phpunit bootstrap="vendor/autoload.php"
         colors="true"
         testdox="true">
    <testsuites>
        <testsuite name="Unit">
            <directory>tests</directory>
        </testsuite>
    </testsuites>
</phpunit>
```

4. Periksa bahwa PHPUnit siap dipakai:

Terminal

```
vendor/bin/phpunit --version
```

Hasil yang diharapkan

Terminal menampilkan PHPUnit 11.5.x. Jika muncul pesan versi PHP tidak memenuhi syarat, perbarui PHP terlebih dahulu.

```
PS C:\Users\PC-26\belajar-phpunit> vendor/bin/phpunit --version
PHPUnit 11.5.0 by Sebastian Bergmann and contributors.
```

Praktikum 2 – Test Pertama: Greeter (25 menit)

1. Buat class yang akan diuji:

src/Greeter.php

```
<?php declare(strict_types=1);

// Class sederhana yang akan diuji: membuat kalimat sapaan
final class Greeter
{
    public function greet(string $name): string
    {
        return 'Hello, ' . $name . '!';
    }
}
```

2. Buat file test. Perhatikan konvensi nama class dan method, serta pola AAA:

tests/GreeterTest.php

```
<?php declare(strict_types=1);

use PHPUnit\Framework\TestCase;

final class GreeterTest extends TestCase
{
    public function testGreetsWithName(): void
    {
        // Arrange: siapkan objek yang diuji
        $greeter = new Greeter();

        // Act: jalankan perilaku yang diuji
        $greeting = $greeter->greet('Alice');

        // Assert: bandingkan hasil dengan harapan
        $this->assertSame('Hello, Alice!', $greeting);
    }
}
```

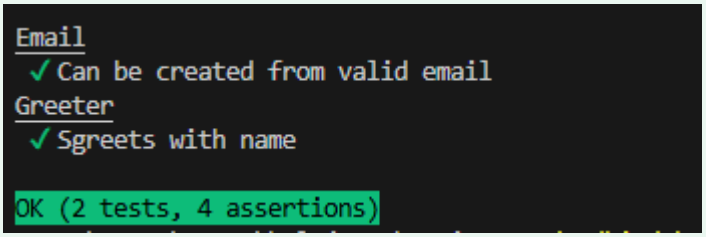
3. Perbarui autoloader karena ada class baru, lalu jalankan test:

Terminal

```
composer dump-autoload
vendor/bin/phpunit
```

Hasil yang diharapkan

Muncul ✓ Greets with name dan baris terakhir OK (1 test, 1 assertion).



4. Di src/Greeter.php, ganti 'Hello, ' menjadi 'Hi, '. Jalankan test lagi dan catat pesan yang muncul. Kembalikan seperti semula.
5. Ganti nama method testGreetsWithName menjadi greetsWithName (tanpa awalan test). Jalankan test, amati hasilnya, lalu kembalikan.

Pertanyaan pengamatan (jawab di laporan)

1. Pada langkah 4, apa arti baris yang diawali tanda - dan + pada pesan gagal?
2. Pada langkah 5, apa yang terjadi? Mengapa awalan test penting?
3. Tunjukkan bagian Arrange, Act, dan Assert pada test yang Anda tulis.

Jawaban

1. Baris bertanda - (*Expected*) menunjukkan nilai harapan yang didefinisikan di dalam test ('Hello, Alice!'), sedangkan baris bertanda + (*Actual*) menunjukkan nilai aktual yang dikembalikan oleh method greet() setelah kode diubah ('Hi, Alice!'). Format *diff* ini memudahkan kita melihat letak perbedaan teks secara langsung.

2. Saat nama method diubah menjadi greetsWithName (tanpa awalan test), PHPUnit tidak menjalankan method tersebut dan menampilkan peringatan No tests executed!. Awalan test penting karena secara *default* PHPUnit mengenali method yang harus dieksekusi sebagai unit test berdasarkan konvensi penamaan berawalan test (kecuali ditandai secara eksplisit dengan atribut #[Test]).

3. Bukti

```
0 references
final class GreeterTest extends TestCase
{
    0 references
    public function testsgreetsWithName(): void
    {
        // Arrange: siapkan objek yang diuji
        $greeter = new Greeter();

        // Act: jalankan perilaku yang diuji
        $greeting = $greeter->greet('Alice');

        // Assert: bandingkan hasil dengan harapan
        $this->assertSame('Hello, Alice!', $greeting);
    }
```

Praktikum 3 – Beberapa Assertion: Email (25 menit)

Class Email hanya bisa dibuat melalui method fromString(), dan akan menolak alamat yang tidak valid. Pada praktikum ini kita hanya menguji alamat yang valid.

1. Buat class Email:

src/Email.php

```
<?php declare(strict_types=1);

// Value object alamat email: hanya bisa dibuat dari alamat yang valid
final class Email
{
    private function __construct(private string $address)
    {
    }

    public static function fromString(string $address): self
    {
        if (filter_var($address, FILTER_VALIDATE_EMAIL) === false) {
            throw new InvalidArgumentException("\"$address\" bukan alamat email yang valid");
        }

        return new self($address);
    }

    public function asString(): string
    {
        return $this->address;
    }
}
```

2. Buat test dengan tiga assertion yang memeriksa satu perilaku yang sama:

tests/EmailTest.php

```
<?php declare(strict_types=1);

use PHPUnit\Framework\TestCase;

final class EmailTest extends TestCase
{
    public function testCanBeCreatedFromValidEmail(): void
    {
        $address = 'user@example.com';

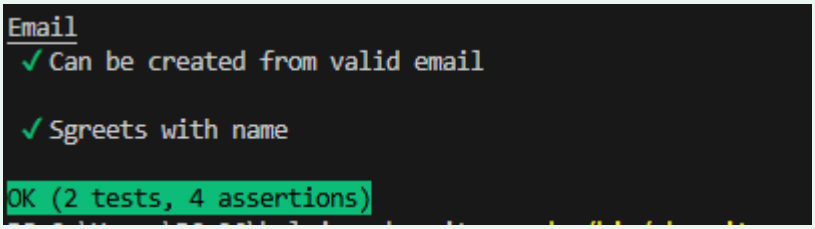
        $email = Email::fromString($address);

        $this->assertInstanceOf(Email::class, $email);
        $this->assertSame($address, $email->asString());
        $this->assertStringContainsString('@', $email->asString());
    }
}
```

3. Jalankan composer dump-autoload, lalu vendor/bin/phpunit.

Hasil yang diharapkan

Terdapat 2 test dari dua class (Greeter dan Email) dan baris terakhir OK (2 tests, 4 assertions).



4. Ubah assertion terakhir menjadi assertStringContainsString('#', ...). Jalankan test, catat hasilnya, lalu kembalikan.
5. Ubah assertSame($address, ...) menjadi assertEquals($address, ...). Apakah hasilnya berbeda? Kembalikan ke assertSame.

Pertanyaan pengamatan (jawab di laporan)

1. Mengapa jumlah assertion pada langkah 3 adalah 4, padahal hanya ada 2 test?
2. Jelaskan perbedaan assertSame dan assertEquals beserta contoh kasus ketika hasil keduanya berbeda.

Jawaban

1. Jumlah *test* dihitung dari banyaknya method test yang dijalankan (testGreetsWithName pada GreeterTest dan testCanBeCreatedFromValidEmail pada EmailTest), sedangkan jumlah *assertion* dihitung dari total pemanggilan method pemeriksaan (assert...). GreeterTest menjalankan 1 assertion dan EmailTest menjalankan 3 assertion (assertInstanceOf, assertSame, dan assertStringContainsString), sehingga totalnya 1 + 3 = 4 assertions.

2. assertSame($expected, $actual) memeriksa kesamaan nilai **sekaligus tipe data** (setara operator identik ===), sedangkan assertEquals($expected, $actual) hanya memeriksa kesamaan nilai dengan mengizinkan konversi tipe otomatis/*type juggling* (setara operator ==).

Praktikum 4 – Data Provider (30 menit)

Daripada menulis banyak method yang hampir sama untuk nama yang berbeda, kita gunakan data provider.

1. Tambahkan baris berikut di bawah <?php declare(strict_types=1); pada tests/GreeterTest.php:

tests/GreeterTest.php (tambahan use)

```
use PHPUnit\Framework\Attributes\DataProvider;
```

2. Tambahkan method berikut di dalam class GreeterTest, setelah testGreetsWithName():

tests/GreeterTest.php (tambahan method)

```
#[DataProvider('nameProvider')]
public function testGreetsEachName(string $name, string $expected): void
{
    $greeter = new Greeter();

    $this->assertSame($expected, $greeter->greet($name));
}

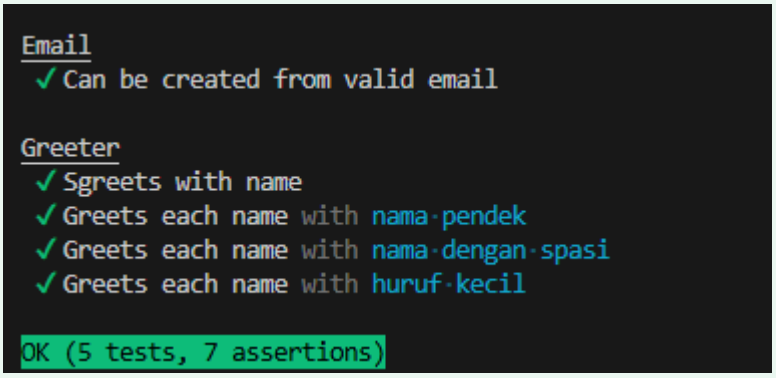
// Data provider: setiap baris = satu kasus uji
public static function nameProvider(): array
{
    return [
        'nama pendek'       => ['Ani', 'Hello, Ani!'],
        'nama dengan spasi' => ['Siti Aminah', 'Hello, Siti Aminah!'],
        'huruf kecil'       => ['dedi', 'Hello, dedi!'],
    ];
}
```

Setiap baris array dipetakan ke parameter method sesuai urutan: elemen pertama ke `$name`, elemen kedua ke `$expected`. Key array (misalnya `'nama pendek'`) menjadi nama kasus di laporan.

3. Jalankan test.

Hasil yang diharapkan

Baris terakhir `OK (5 tests, 7 assertions)`: 1 test Greeter biasa, 3 kasus data provider, dan 1 test Email.



4. Tambahkan baris yang sengaja salah ke `nameProvider()`: `'sengaja salah' => ['Dedi', 'Halo, Dedi!'],`. Jalankan test dan baca laporan kegagalannya.
5. Hapus kembali baris tersebut dan pastikan semua test lulus.

Pertanyaan pengamatan (jawab di laporan)

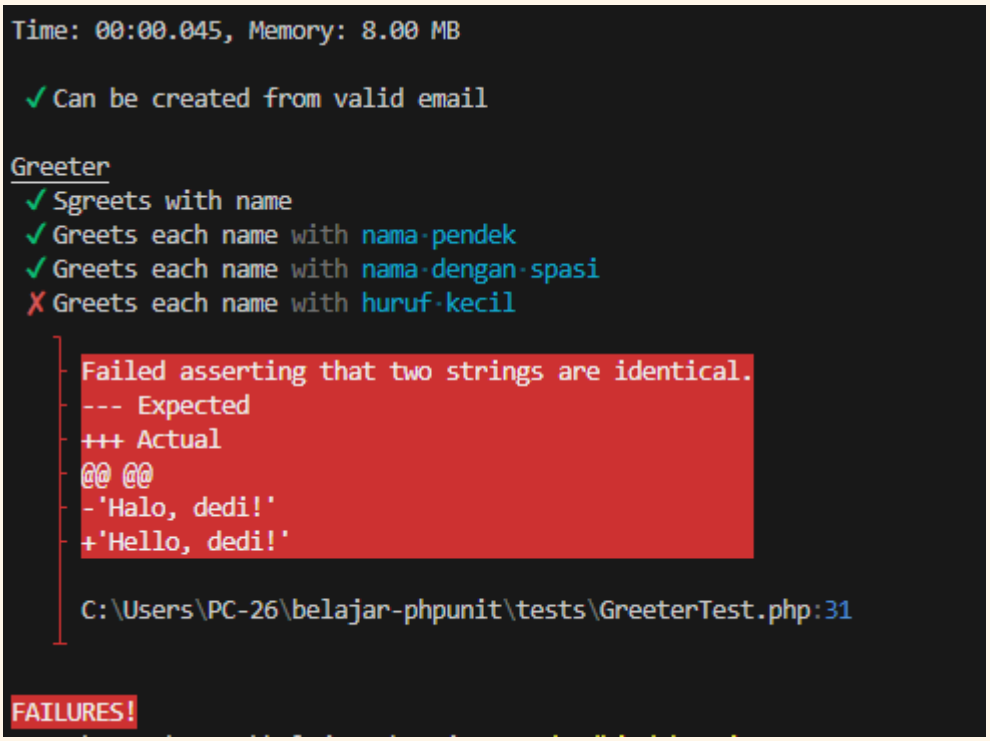
1. Bagaimana PHPUnit menunjukkan baris data mana yang gagal pada langkah 4?
2. Pada langkah 4, apakah yang salah kode Greeter atau data ujinya? Bagaimana Anda memutuskannya?
3. Tambahkan satu baris data baru yang menurut Anda menarik untuk diuji. Jelaskan alasannya.

Jawaban

1. PHPUnit mencantumkan nama method beserta *key* (label) dari elemen array data provider pada pesan kegagalan dan daftar *TestDox*, misalnya: GreeterTest::testGreetsEachName with data set "sengaja salah" (atau ✗ Greets each name with sengaja salah).

2. Yang salah adalah **data ujinya** ('Halo, Dedi!'). Cara memutuskannya adalah dengan memeriksa spesifikasi/kebutuhan sistem: class Greeter memang dirancang untuk menyapa dalam bahasa Inggris ('Hello, <nama>!'), dan seluruh kasus uji lainnya konsisten mengharapkan 'Hello, ...', bukan 'Halo, ...'.

3. Menguji nama yang mengandung tanda petik tunggal (') dan tanda hubung (-) memastikan method greet() tidak merusak, memotong, atau melakukan *escaping* yang tidak diinginkan pada karakter spesial di dalam nama.



E. Latihan (45 menit)

Latihan 1 – Data provider untuk Email

Di EmailTest, buat method test baru yang memakai data provider untuk memastikan minimal **4 alamat email valid** yang berbeda bentuknya dapat dibuat dan dikembalikan apa adanya oleh asString(). Contoh variasi: alamat dengan titik di nama, angka, subdomain, atau tanda plus.

Latihan 2 – Siapa yang menangkap bug?

Ganti isi method greet() satu per satu dengan versi berikut. Untuk setiap versi, jalankan test dan catat test atau kasus data provider mana yang gagal. Kembalikan kode asli setelah selesai.

Versi	Isi baris return
A	return 'Hello ' . $name . '!'; (koma hilang)
B	return 'Hello, ' . ucfirst($name) . '!'; (huruf awal dijadikan kapital)
C	return 'Hello, ' . trim($name) . '!'; (spasi di tepi dibuang)

Pertanyaan pengamatan (jawab di laporan)

1. Versi mana yang tidak tertangkap oleh test yang ada? Mengapa?
2. Tambahkan satu baris data ke nameProvider() agar versi tersebut tertangkap.

Jawaban

1. **Versi C** (trim($name)) tidak tertangkap karena seluruh data uji yang ada ('Alice', 'Ani', 'Siti Aminah', 'dedi') hanya memiliki spasi di tengah kata atau tanpa spasi sama sekali, dan tidak ada yang memiliki spasi di awal atau di akhir string. Akibatnya, trim($name) menghasilkan teks yang identik dengan $name.
2. **Bukti**

```
Time: 00:00.051, Memory: 8.00 MB

Email
 ✓ Can be created from valid email
 ✓ Can be created from various valid emails with titik·di·nama
 ✓ Can be created from various valid emails with mengandung·angka
 ✓ Can be created from various valid emails with menggunakan·subdomain

Greeter
 ✓ Sgreets with name
 ✓ Greets each name with nama·pendek
 ✓ Greets each name with nama·dengan·spasi
 ✓ Greets each name with huruf·kecil
 ✗ Greets each name with spasi·di·tepi

   Failed asserting that two strings are identical.
   --- Expected
   +++ Actual
   @@ @@
   -'Hello,  Ani !'
   +'Hello, Ani!'

   C:\Users\PC-26\belajar-phpunit\tests\GreeterTest.php:31
```

```php
// Data provider: setiap baris = satu kasus uji
1 reference
public static function nameProvider(): array
{
    return [
        'nama pendek'       => ['Ani', 'Hello, Ani!'],
        'nama dengan spasi' => ['Siti Aminah', 'Hello, Siti Aminah!'],
        'huruf kecil'       => ['dedi', 'Hello, dedi!'],
        'spasi di tepi' => [' Ani ', 'Hello,  Ani !'],
    ];
}
```

F. Tugas Mandiri

Buat class KonversiSuhu di folder src dengan dua method berikut, lalu tulis test-nya di tests/KonversiSuhuTest.php:

Method	Rumus
celsiusKeFahrenheit(float $c): float	F = C × 9/5 + 32
celsiusKeKelvin(float $c): float	K = C + 273,15

- Gunakan data provider dengan minimal 4 kasus untuk setiap method, misalnya titik beku, titik didih, suhu negatif, dan suhu tubuh.
- Semua test memakai pola AAA dan nama method yang menjelaskan perilaku.
- Jalankan vendor/bin/phpunit dan pastikan semua test lulus.

Petunjuk

Coba uji 36,6 °C dengan assertSame(97.88, ...). Jika gagal padahal rumus benar, cari tahu penyebabnya, lalu gunakan assertEqualsWithDelta(97.88, $hasil, 0.001). Jelaskan temuan ini di laporan.

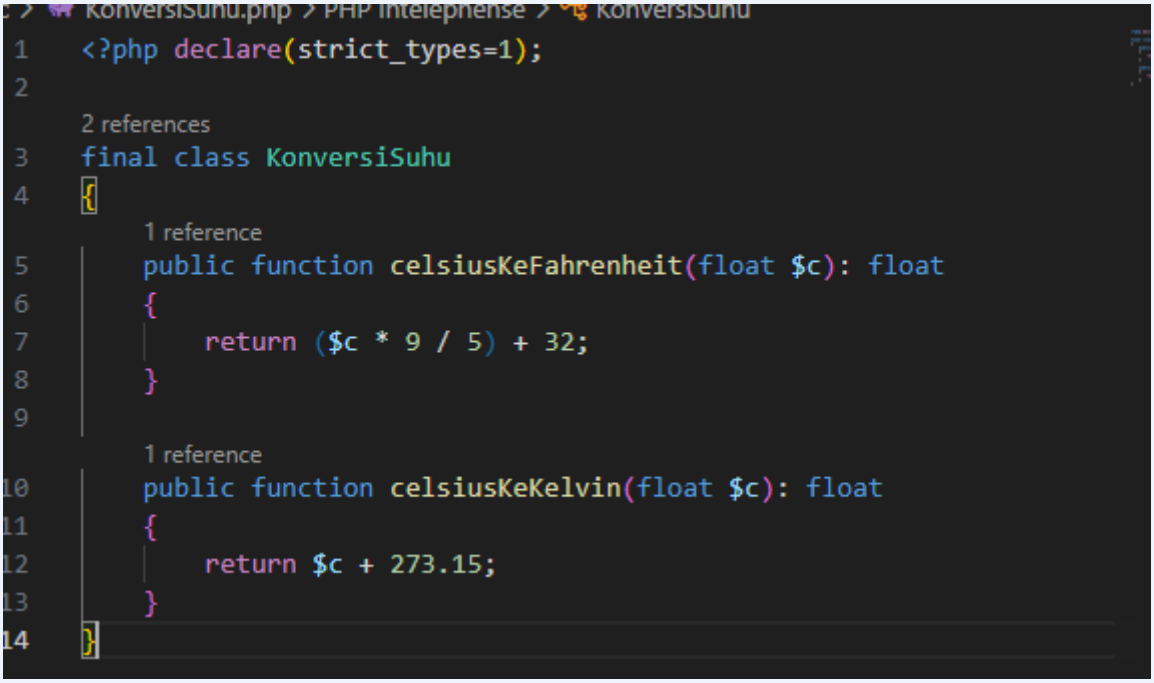
Jawaban

1. **Penyebab kegagalan assertSame(97.88, $hasil)**

   Dalam representasi bilangan pecahan biner standar IEEE 754 (*floating-point*), angka desimal seperti 36.6 tidak dapat disimpan secara eksak di memori komputer. Saat operasi 36.6 * 9 / 5 + 32 dihitung oleh PHP, hasilnya adalah 97.88000000000001, bukan tepat 97.88. Karena assertSame() menuntut kesamaan bit secara mutlak tanpa toleransi, pengujian menjadi gagal (Failed asserting that 97.88000000000001 is identical to 97.88).

2. **Solusi dengan assertEqualsWithDelta(97.88, $hasil, 0.001):** Method assertEqualsWithDelta() memeriksa apakah selisih absolut antara nilai harapan dan nilai aktual masih berada di dalam batas toleransi (*delta*) 0.001 ($|97.88 - 97.88000000000001| \leq 0.001$). Karena selisihnya sangat kecil (10^{-14}), assertion dinyatakan lulus. Untuk operasi *floating-point*, assertEqualsWithDelta() selalu direkomendasikan dibanding assertSame().

3. Bukti



tests > KonversiSuhuTest.php > PHP Intelephense > KonversiSuhuTest

```php
<?php declare(strict_types=1);

use PHPUnit\Framework\Attributes\DataProvider;
use PHPUnit\Framework\TestCase;

0 references
final class KonversiSuhuTest extends TestCase
{
    #[DataProvider('celsiusToFahrenheitProvider')]
    0 references
    public function testConvertsCelsiusToFahrenheitCorrectly(float $celsius,
    {
        // Arrange
        $konversi = new KonversiSuhu();

        // Act
        $hasil = $konversi->celsiusKeFahrenheit($celsius);

        // Assert
        $this->assertEqualsWithDelta($expectedFahrenheit, $hasil, 0.001);
    }

    1 reference
    public static function celsiusToFahrenheitProvider(): array
    {
        return [
            'titik beku air'  => [0.0, 32.0],
            'titik didih air' => [100.0, 212.0],
            'suhu negatif'    => [-40.0, -40.0],
            'suhu tubuh'      => [36.6, 97.88],
        ];
    }
```

```
Konversi Suhu
 ✓ Converts celsius to fahrenheit correctly with titik beku air
 ✓ Converts celsius to fahrenheit correctly with titik didih air
 ✓ Converts celsius to fahrenheit correctly with suhu negatif
 ✓ Converts celsius to fahrenheit correctly with suhu tubuh
 ✓ Converts celsius to kelvin correctly with titik beku air
 ✓ Converts celsius to kelvin correctly with titik didih air
 ✓ Converts celsius to kelvin correctly with suhu nol mutlak
 ✓ Converts celsius to kelvin correctly with suhu tubuh
```

KESIMPULAN

Melalui praktikum ini, dapat disimpulkan bahwa PHPUnit mempermudah pengujian unit (*unit testing*) secara terstruktur di PHP menggunakan pola **Arrange-Act-Assert (AAA)** untuk memisahkan tahap persiapan objek, eksekusi method, dan verifikasi hasil. Pemilihan jenis *assertion* sangat menentukan ketelitian pengujian: assertSame() memastikan kesamaan nilai dan tipe data secara ketat, sedangkan

assertEqualsWithDelta() diperlukan saat menguji perhitungan *floating-point* guna menghindari kegagalan akibat ketidakakuratan presisi biner IEEE 754. Selain itu, penggunaan atribut #[DataProvider] memungkinkan satu method test dijalankan berulang kali dengan berbagai variasi kasus uji secara ringkas dan mudah dipelihara.

G. Laporan

Kumpulkan laporan dalam format PDF yang berisi:

1. Tautan repositori Git berisi kode praktikum, latihan, dan tugas.
2. Tangkapan layar hasil vendor/bin/phpunit di akhir setiap praktikum dan tugas.
3. Jawaban seluruh pertanyaan pengamatan.
4. Kesimpulan singkat tentang apa yang Anda pelajari.

H. Referensi

Bergmann, S. PHPUnit Manual: Writing Tests for PHPUnit. https://docs.phpunit.de/

PHPUnit. Getting Started with PHPUnit. https://phpunit.de/getting-started/